

计算几何

计算几何模板

```
#include <bits/stdc++.h>
using namespace std;

using point_t=long double; //全局数据类型，可修改为 long long 等

constexpr point_t eps=1e-8;
constexpr long double PI=3.14159265358979323841;

// 点与向量
template<typename T> struct point
{
    T x,y;

    bool operator==(const point &a) const {return (abs(x-a.x)<=eps && abs(y-a.y)<=eps);}
    bool operator<(const point &a) const {if (abs(x-a.x)<=eps) return y<a.y-eps; return x<a.x-eps;}
    bool operator>(const point &a) const {return !(*this<a || *this==a);}
    point operator+(const point &a) const {return {x+a.x,y+a.y};}
    point operator-(const point &a) const {return {x-a.x,y-a.y};}
    point operator-() const {return {-x,-y};}
    point operator*(const T k) const {return {k*x,k*y};}
    point operator/(const T k) const {return {x/k,y/k};}
    T operator*(const point &a) const {return x*a.x+y*a.y;} // 点积
    T operator^(const point &a) const {return x*a.y-y*a.x;} // 叉积，注意优先级
    int toleft(const point &a) const {const auto t=(*this)^a; return (t>eps)-(t<-eps);} // to-left
    T len2() const {return (*this)*(*this);} // 向量长度的平方
    T dis2(const point &a) const {return (a-(*this)).len2();} // 两点距离的平方

    // 涉及浮点数
    long double len() const {return sqrtl(len2());} // 向量长度
    long double dis(const point &a) const {return sqrtl(dis2(a));} // 两点距离
    long double ang(const point &a) const {return acosl(max(-1.01,min(1.01,((*this)*a)/(len()*a.len())));}
    point rot(const long double rad) const {return {x*cos(rad)-y*sin(rad),x*sin(rad)+y*cos(rad)};
    point rot(const long double cosr,const long double sinr) const {return {x*cosr-y*sinr,x*sinr+y*cosr};}
};

using Point=point<point_t>;
```

```

// 极角排序
struct argcmp
{
    bool operator()(const Point &a,const Point &b) const
    {
        const auto quad=[](const Point &a)
        {
            if (a.y<-eps) return 1;
            if (a.y>eps) return 4;
            if (a.x<-eps) return 5;
            if (a.x>eps) return 3;
            return 2;
        };
        const int qa=quad(a),qb=quad(b);
        if (qa!=qb) return qa<qb;
        const auto t=a^b;
        return t>eps;
    }
};

// 直线
template<typename T> struct line
{
    point<T> p,v; // p 为直线上一点, v 为方向向量

    bool operator==(const line &a) const {return v.toleft(a.v)==0 && v.toleft(p-a.p)==0;}
    int toleft(const point<T> &a) const {return v.toleft(a-p);} // to-left 测试
    bool operator<(const line &a) const // 半平面交算法定义的排序
    {
        if (abs(v^a.v)<=eps && v*a.v>=-eps) return toleft(a.p)==-1;
        return argcmp()(v,a.v);
    }

    // 涉及浮点数
    point<T> inter(const line &a) const {return p+v*((a.v^(p-a.p))/(v^a.v));} // 直线交点
    long double dis(const point<T> &a) const {return abs(v^(a-p))/v.len();} // 点到直线距离
    point<T> proj(const point<T> &a) const {return p+v*((v*(a-p))/(v*v));} // 点在直线上的投影
};

using Line=line<point_t>;

//线段
template<typename T> struct segment

```

```

{
    point<T> a,b;

    bool operator<(const segment &s) const {return make_pair(a,b)<make_pair(s.a,s.b);}

    // 判定性函数建议在整数域使用

    // 判断点是否在线段上
    // -1 点在线段端点 | 0 点不在线段上 | 1 点严格在线段上
    int is_on(const point<T> &p) const
    {
        if (p==a || p==b) return -1;
        return (p-a).toleft(p-b)==0 && (p-a)*(p-b)<-eps;
    }

    // 判断线段直线是否相交
    // -1 直线经过线段端点 | 0 线段和直线不相交 | 1 线段和直线严格相交
    int is_inter(const line<T> &l) const
    {
        if (l.toleft(a)==0 || l.toleft(b)==0) return -1;
        return l.toleft(a)!=l.toleft(b);
    }

    // 判断两线段是否相交
    // -1 在某一线段端点处相交 | 0 两线段不相交 | 1 两线段严格相交
    int is_inter(const segment<T> &s) const
    {
        if (is_on(s.a) || is_on(s.b) || s.is_on(a) || s.is_on(b)) return -1;
        const line<T> l{a,b-a},ls{s.a,s.b-s.a};
        return l.toleft(s.a)*l.toleft(s.b)==-1 && ls.toleft(a)*ls.toleft(b)==-1;
    }

    // 点到线段距离
    long double dis(const point<T> &p) const
    {
        if ((p-a)*(b-a)<-eps || (p-b)*(a-b)<-eps) return min(p.dis(a),p.dis(b));
        const line<T> l{a,b-a};
        return l.dis(p);
    }

    // 两线段间距离
    long double dis(const segment<T> &s) const
    {

```

```

        if (is_inter(s)) return 0;
        return min({dis(s.a),dis(s.b),s.dis(a),s.dis(b)});
    }
};

```

```

using Segment=segment<point_t>;

```

```

// 多边形

```

```

template<typename T> struct polygon

```

```

{

```

```

    vector<point<T>> p; // 以逆时针顺序存储

```

```

    size_t nxt(const size_t i) const {return i==p.size()-1?0:i+1;}

```

```

    size_t pre(const size_t i) const {return i==0?p.size()-1:i-1;}

```

```

// 回转数

```

```

// 返回值第一项表示点是否在多边形边上

```

```

// 对于狭义多边形，回转数为 0 表示点在多边形外，否则点在多边形内

```

```

pair<bool,int> winding(const point<T> &a) const

```

```

{

```

```

    int cnt=0;

```

```

    for (size_t i=0;i<p.size();i++)

```

```

    {

```

```

        const point<T> u=p[i],v=p[nxt(i)];

```

```

        if (abs((a-u)^(a-v))<=eps && (a-u)*(a-v)<=eps) return {true,0};

```

```

        if (abs(u.y-v.y)<=eps) continue;

```

```

        const Line uv={u,v-u};

```

```

        if (u.y<v.y-eps && uv.toleft(a)<=0) continue;

```

```

        if (u.y>v.y+eps && uv.toleft(a)>=0) continue;

```

```

        if (u.y<a.y-eps && v.y>=a.y-eps) cnt++;

```

```

        if (u.y>=a.y-eps && v.y<a.y-eps) cnt--;

```

```

    }

```

```

    return {false,cnt};

```

```

}

```

```

// 多边形面积的两倍

```

```

// 可用于判断点的存储顺序是顺时针或逆时针

```

```

T area() const

```

```

{

```

```

    T sum=0;

```

```

    for (size_t i=0;i<p.size();i++) sum+=p[i]^p[nxt(i)];

```

```

    return sum;

```

```

}

```

```

// 多边形的周长
long double circ() const
{
    long double sum=0;
    for (size_t i=0;i<p.size();i++) sum+=p[i].dis(p[nxt(i)]);
    return sum;
}

};

using Polygon=polygon<point_t>;

//凸多边形
template<typename T> struct convex: polygon<T>
{
    // 闵可夫斯基和
    convex operator+(const convex &c) const
    {
        const auto &p=this->p;
        vector<Segment> e1(p.size()),e2(c.p.size()),edge(p.size()+c.p.size());
        vector<point<T>> res; res.reserve(p.size()+c.p.size());
        const auto cmp=[](const Segment &u,const Segment &v) {return argcmp()(u.b-u.a,v.b-v.a);};
        for (size_t i=0;i<p.size();i++) e1[i]={p[i],p[this->nxt(i)]};
        for (size_t i=0;i<c.p.size();i++) e2[i]={c.p[i],c.p[c.nxt(i)]};
        rotate(e1.begin(),min_element(e1.begin(),e1.end(),cmp),e1.end());
        rotate(e2.begin(),min_element(e2.begin(),e2.end(),cmp),e2.end());
        merge(e1.begin(),e1.end(),e2.begin(),e2.end(),edge.begin(),cmp);
        const auto check=[](const vector<point<T>> &res,const point<T> &u)
        {
            const auto back1=res.back(),back2=*prev(res.end(),2);
            return (back1-back2).toleft(u-back1)==0 && (back1-back2)*(u-back1)>=-eps;
        };
        auto u=e1[0].a+e2[0].a;
        for (const auto &v:edge)
        {
            while (res.size()>1 && check(res,u)) res.pop_back();
            res.push_back(u);
            u=u+v.b-v.a;
        }
        if (res.size()>1 && check(res,res[0])) res.pop_back();
        return {res};
    }
}

```

```

// 旋转卡壳
// func 为更新答案的函数，可以根据题目调整位置
template<typename F> void rotcaliper(const F &func) const
{
    const auto &p=this->p;
    const auto area=[&](const point<T> &u,const point<T> &v,const point<T> &w){return (w-u)^v;}
    for (size_t i=0,j=1;i<p.size();i++)
    {
        const auto nxti=this->nxt(i);
        func(p[i],p[nxti],p[j]);
        while (area(p[this->nxt(j)],p[i],p[nxti])>=area(p[j],p[i],p[nxti]))
        {
            j=this->nxt(j);
            func(p[i],p[nxti],p[j]);
        }
    }
}

// 凸多边形的直径的平方
T diameter2() const
{
    const auto &p=this->p;
    if (p.size()==1) return 0;
    if (p.size()==2) return p[0].dis2(p[1]);
    T ans=0;
    auto func=[&](const point<T> &u,const point<T> &v,const point<T> &w){ans=max({ans,w.dis2(u),v.dis2(u),v.dis2(w)});return 0;}
    rotcaliper(func);
    return ans;
}

// 判断点是否在凸多边形内
// 复杂度 O(logn)
// -1 点在多边形边上 | 0 点在多边形外 | 1 点在多边形内
int is_in(const point<T> &a) const
{
    const auto &p=this->p;
    if (p.size()==1) return a==p[0]?-1:0;
    if (p.size()==2) return segment<T>{p[0],p[1]}.is_on(a)?-1:0;
    if (a==p[0]) return -1;
    if ((p[1]-p[0]).toleft(a-p[0])==-1 || (p.back()-p[0]).toleft(a-p[0])==1) return 0;
    const auto cmp=[&](const Point &u,const Point &v){return (u-p[0]).toleft(v-p[0])>0;}
    const size_t i=lower_bound(p.begin()+1,p.end(),a,cmp)-p.begin();
    if (i==1) return segment<T>{p[0],p[i]}.is_on(a)?-1:0;
}

```

```

    if (i==p.size()-1 && segment<T>{p[0],p[i]}.is_on(a)) return -1;
    if (segment<T>{p[i-1],p[i]}.is_on(a)) return -1;
    return (p[i]-p[i-1]).toleft(a-p[i-1])>0;
}

```

// 凸多边形关于某一方向的极点

// 复杂度 $O(\log n)$

// 参考资料: <https://codeforces.com/blog/entry/48868>

```

template<typename F> size_t extreme(const F &dir) const
{
    const auto &p=this->p;
    const auto check=[&](const size_t i){return dir(p[i]).toleft(p[this->nxt(i)]-p[i])>=0;};
    const auto dir0=dir(p[0]); const auto check0=check(0);
    if (!check0 && check(p.size()-1)) return 0;
    const auto cmp=[&](const Point &v)
    {
        const size_t vi=&v-p.data();
        if (vi==0) return 1;
        const auto checkv=check(vi);
        const auto t=dir0.toleft(v-p[0]);
        if (vi==1 && checkv==check0 && t==0) return 1;
        return checkv^(checkv==check0 && t<=0);
    };
    return partition_point(p.begin(),p.end(),cmp)-p.begin();
}

```

// 过凸多边形外一点求凸多边形的切线, 返回切点下标

// 复杂度 $O(\log n)$

// 必须保证点在多边形外

```

pair<size_t,size_t> tangent(const point<T> &a) const
{
    const size_t i=extreme([&](const point<T> &u){return u-a;});
    const size_t j=extreme([&](const point<T> &u){return a-u;});
    return {i,j};
}

```

// 求平行于给定直线的凸多边形的切线, 返回切点下标

// 复杂度 $O(\log n)$

```

pair<size_t,size_t> tangent(const line<T> &a) const
{
    const size_t i=extreme([&](...){return a.v;});
    const size_t j=extreme([&](...){return -a.v;});
    return {i,j};
}

```

```

    }
};

using Convex=convex<point_t>;

// 圆
struct Circle
{
    Point c;
    long double r;

    bool operator==(const Circle &a) const {return c==a.c && abs(r-a.r)<=eps;}
    long double circ() const {return 2*PI*r;} // 周长
    long double area() const {return PI*r*r;} // 面积

    // 点与圆的关系
    // -1 圆上 | 0 圆外 | 1 圆内
    int is_in(const Point &p) const {const long double d=p.dis(c); return abs(d-r)<=eps?-1:d<r-eps?0:1;}

    // 直线与圆关系
    // 0 相离 | 1 相切 | 2 相交
    int relation(const Line &l) const
    {
        const long double d=l.dis(c);
        if (d>r+eps) return 0;
        if (abs(d-r)<=eps) return 1;
        return 2;
    }

    // 圆与圆关系
    // -1 相同 | 0 相离 | 1 外切 | 2 相交 | 3 内切 | 4 内含
    int relation(const Circle &a) const
    {
        if (*this==a) return -1;
        const long double d=c.dis(a.c);
        if (d>r+a.r+eps) return 0;
        if (abs(d-r-a.r)<=eps) return 1;
        if (abs(d-abs(r-a.r))<=eps) return 3;
        if (d<abs(r-a.r)-eps) return 4;
        return 2;
    }

    // 直线与圆的交点

```



```

vector<Point> inter(const Line &l) const
{
    const long double d=l.dis(c);
    const Point p=l.proj(c);
    const int t=relation(l);
    if (t==0) return vector<Point>();
    if (t==1) return vector<Point>{p};
    const long double k=sqrt(r*r-d*d);
    return vector<Point>{p-(l.v/l.v.len())*k,p+(l.v/l.v.len())*k};
}

```

// 圆与圆交点

```

vector<Point> inter(const Circle &a) const
{
    const long double d=c.dis(a.c);
    const int t=relation(a);
    if (t== -1 || t==0 || t==4) return vector<Point>();
    Point e=a.c-c; e=e/e.len()*r;
    if (t==1 || t==3)
    {
        if (r*r+d*d-a.r*a.r>=-eps) return vector<Point>{c+e};
        return vector<Point>{c-e};
    }
    const long double costh=(r*r+d*d-a.r*a.r)/(2*r*d),sinh=sqrt(1-costh*costh);
    return vector<Point>{c+e.rot(costh,-sinh),c+e.rot(costh,sinh)};
}

```

// 圆与圆交面积

```

long double inter_area(const Circle &a) const
{
    const long double d=c.dis(a.c);
    const int t=relation(a);
    if (t== -1) return area();
    if (t<2) return 0;
    if (t>2) return min(area(),a.area());
    const long double costh1=(r*r+d*d-a.r*a.r)/(2*r*d),costh2=(a.r*a.r+d*d-r*r)/(2*a.r*d);
    const long double sinh1=sqrt(1-costh1*costh1),sinh2=sqrt(1-costh2*costh2);
    const long double th1=acos(costh1),th2=acos(costh2);
    return r*r*(th1-costh1*sinh1)+a.r*a.r*(th2-costh2*sinh2);
}

```

// 过圆外一点圆的切线

```

vector<Line> tangent(const Point &a) const

```

```

{
    const int t=is_in(a);
    if (t==1) return vector<Line>();
    if (t==-1)
    {
        const Point v={-(a-c).y,(a-c).x};
        return vector<Line>{{a,v}};
    }
    Point e=a-c; e=e/e.len()*r;
    const long double costh=r/c.dis(a),sinth=sqrt(1-costh*costh);
    const Point t1=c+e.rot(costh,-sinth),t2=c+e.rot(costh,sinth);
    return vector<Line>{{a,t1-a},{a,t2-a}};
}

// 两圆的公切线
vector<Line> tangent(const Circle &a) const
{
    const int t=relation(a);
    vector<Line> lines;
    if (t==-1 || t==4) return lines;
    if (t==1 || t==3)
    {
        const Point p=inter(a)[0],v={-(a.c-c).y,(a.c-c).x};
        lines.push_back({p,v});
    }
    const long double d=c.dis(a.c);
    const Point e=(a.c-c)/(a.c-c).len();
    if (t<=2)
    {
        const long double costh=(r-a.r)/d,sinth=sqrt(1-costh*costh);
        const Point d1=e.rot(costh,-sinth),d2=e.rot(costh,sinth);
        const Point u1=c+d1*r,u2=c+d2*r,v1=a.c+d1*a.r,v2=a.c+d2*a.r;
        lines.push_back({u1,v1-u1}); lines.push_back({u2,v2-u2});
    }
    if (t==0)
    {
        const long double costh=(r+a.r)/d,sinth=sqrt(1-costh*costh);
        const Point d1=e.rot(costh,-sinth),d2=e.rot(costh,sinth);
        const Point u1=c+d1*r,u2=c+d2*r,v1=a.c-d1*a.r,v2=a.c-d2*a.r;
        lines.push_back({u1,v1-u1}); lines.push_back({u2,v2-u2});
    }
    return lines;
}

```

// 圆的反演

```
tuple<int,Circle,Line> inverse(const Line &l) const
{
    const Circle null_c={{0.0,0.0},0.0};
    const Line null_l={{0.0,0.0},{0.0,0.0}};
    if (l.toleft(c)==0) return {2,null_c,l};
    const Point v=l.toleft(c)==1?Point{l.v.y,-l.v.x}:Point{-l.v.y,l.v.x};
    const long double d=r*r/l.dis(c);
    const Point p=c+v/v.len()*d;
    return {1,{(c+p)/2,d/2},null_l};
}
```

```
tuple<int,Circle,Line> inverse(const Circle &a) const
```

```
{
    const Circle null_c={{0.0,0.0},0.0};
    const Line null_l={{0.0,0.0},{0.0,0.0}};
    const Point v=a.c-c;
    if (a.is_in(c)==-1)
    {
        const long double d=r*r/(a.r+a.r);
        const Point p=c+v/v.len()*d;
        return {2,null_c,{p,{-v.y,v.x}}};
    }
    if (c==a.c) return {1,{c,r*r/a.r},null_l};
    const long double d1=r*r/(c.dis(a.c)-a.r),d2=r*r/(c.dis(a.c)+a.r);
    const Point p=c+v/v.len()*d1,q=c+v/v.len()*d2;
    return {1,{(p+q)/2,p.dis(q)/2},null_l};
}
};
```

// 圆与多边形面积交

```
long double area_inter(const Circle &circ,const Polygon &poly)
```

```
{
    const auto cal=[](const Circle &circ,const Point &a,const Point &b)
    {
        if ((a-circ.c).toleft(b-circ.c)==0) return 0.01;
        const auto ina=circ.is_in(a),inb=circ.is_in(b);
        const Line ab={a,b-a};
        if (ina && inb) return ((a-circ.c)^(b-circ.c))/2;
        if (ina && !inb)
        {
            const auto t=circ.inter(ab);
```

```

        const Point p=t.size()==1?t[0]:t[1];
        const long double ans=((a-circ.c)^(p-circ.c))/2;
        const long double th=(p-circ.c).ang(b-circ.c);
        const long double d=circ.r*circ.r*th/2;
        if ((a-circ.c).toleft(b-circ.c)==1) return ans+d;
        return ans-d;
    }
    if (!ina && inb)
    {
        const Point p=circ.inter(ab)[0];
        const long double ans=((p-circ.c)^(b-circ.c))/2;
        const long double th=(a-circ.c).ang(p-circ.c);
        const long double d=circ.r*circ.r*th/2;
        if ((a-circ.c).toleft(b-circ.c)==1) return ans+d;
        return ans-d;
    }
    const auto p=circ.inter(ab);
    if (p.size()==2 && Segment{a,b}.dis(circ.c)<=circ.r+eps)
    {
        const long double ans=((p[0]-circ.c)^(p[1]-circ.c))/2;
        const long double th1=(a-circ.c).ang(p[0]-circ.c),th2=(b-circ.c).ang(p[1]-circ.c);
        const long double d1=circ.r*circ.r*th1/2,d2=circ.r*circ.r*th2/2;
        if ((a-circ.c).toleft(b-circ.c)==1) return ans+d1+d2;
        return ans-d1-d2;
    }
    const long double th=(a-circ.c).ang(b-circ.c);
    if ((a-circ.c).toleft(b-circ.c)==1) return circ.r*circ.r*th/2;
    return -circ.r*circ.r*th/2;
};

long double ans=0;
for (size_t i=0;i<poly.p.size();i++)
{
    const Point a=poly.p[i],b=poly.p[poly.nxt(i)];
    ans+=cal(circ,a,b);
}
return ans;
}

// 点集的凸包
// Andrew 算法, 复杂度 O(nlogn)
Convex convexhull(vector<Point> p)
{

```

```

vector<Point> st;
if (p.empty()) return Convex{st};
sort(p.begin(),p.end());
const auto check=[](const vector<Point> &st,const Point &u)
{
    const auto back1=st.back(),back2=*prev(st.end(),2);
    return (back1-back2).toleft(u-back1)<=0;
};
for (const Point &u:p)
{
    while (st.size()>1 && check(st,u)) st.pop_back();
    st.push_back(u);
}
size_t k=st.size();
p.pop_back(); reverse(p.begin(),p.end());
for (const Point &u:p)
{
    while (st.size()>k && check(st,u)) st.pop_back();
    st.push_back(u);
}
st.pop_back();
return Convex{st};
}

```

// 半平面交

// 排序增量法，复杂度 $O(n\log n)$

// 输入与返回值都是用直线表示的半平面集合

```

vector<Line> halfinter(vector<Line> l, const point_t lim=1e9)
{
    const auto check=[](const Line &a,const Line &b,const Line &c){return a.toleft(b.inter(c))<0;};
    l.push_back({{-lim,0},{0,-1}}); l.push_back({{0,-lim},{1,0}});
    l.push_back({{lim,0},{0,1}}); l.push_back({{0,lim},{-1,0}});
    sort(l.begin(),l.end());
    deque<Line> q;
    for (size_t i=0;i<l.size();i++)
    {
        if (i>0 && l[i-1].v.toleft(l[i].v)==0 && l[i-1].v*l[i].v>eps) continue;
        while (q.size()>1 && check(l[i],q.back(),q[q.size()-2])) q.pop_back();
        while (q.size()>1 && check(l[i],q[0],q[1])) q.pop_front();
        if (!q.empty() && q.back().v.toleft(l[i].v)<=0) return vector<Line>();
        q.push_back(l[i]);
    }
    while (q.size()>1 && check(q[0],q.back(),q[q.size()-2])) q.pop_back();
}

```

```

    while (q.size() > 1 && check(q.back(), q[0], q[1])) q.pop_front();
    return vector<Line>(q.begin(), q.end());
}

// 点集形成的最小最大三角形
// 极角序扫描线, 复杂度  $O(n^2 \log n)$ 
// 最大三角形问题可以使用凸包与旋转卡壳做到  $O(n^2)$ 
pair<point_t, point_t> minmax_triangle(const vector<Point> &vec)
{
    if (vec.size() <= 2) return {0, 0};
    vector<pair<int, int>> evt;
    evt.reserve(vec.size() * vec.size());
    point_t maxans = 0, minans = numeric_limits<point_t>::max();
    for (size_t i = 0; i < vec.size(); i++)
    {
        for (size_t j = 0; j < vec.size(); j++)
        {
            if (i == j) continue;
            if (vec[i] == vec[j]) minans = 0;
            else evt.push_back({i, j});
        }
    }
    sort(evt.begin(), evt.end(), [&](const pair<int, int> &u, const pair<int, int> &v)
    {
        const Point du = vec[u.second] - vec[u.first], dv = vec[v.second] - vec[v.first];
        return argcmp()({du.y, -du.x}, {dv.y, -dv.x});
    });
    vector<size_t> vx(vec.size()), pos(vec.size());
    for (size_t i = 0; i < vec.size(); i++) vx[i] = i;
    sort(vx.begin(), vx.end(), [&](int x, int y) { return vec[x] < vec[y]; });
    for (size_t i = 0; i < vx.size(); i++) pos[vx[i]] = i;
    for (auto [u, v]: evt)
    {
        const size_t i = pos[u], j = pos[v];
        const size_t l = min(i, j), r = max(i, j);
        const Point vecu = vec[u], vecv = vec[v];
        if (l > 0) minans = min(minans, abs((vec[vx[l-1]] - vecu)^(vec[vx[l-1]] - vecv)));
        if (r < vx.size() - 1) minans = min(minans, abs((vec[vx[r+1]] - vecu)^(vec[vx[r+1]] - vecv)));
        maxans = max({maxans, abs((vec[vx[0]] - vecu)^(vec[vx[0]] - vecv)), abs((vec[vx.back()] - vecu)^(
            if (i < j) swap(vx[i], vx[j]), pos[u] = j, pos[v] = i;
        }
    }
    return {minans, maxans};
}

```

```

// 判断多条线段是否有交点
// 扫描线，复杂度  $O(n\log n)$ 
bool segs_inter(const vector<Segment> &segs)
{
    if (segs.empty()) return false;
    using seq_t=tuple<point_t,int,Segment>;
    const auto seqcmp=[](const seq_t &u, const seq_t &v)
    {
        const auto [u0,u1,u2]=u;
        const auto [v0,v1,v2]=v;
        if (abs(u0-v0)<=eps) return make_pair(u1,u2)<make_pair(v1,v2);
        return u0<v0-eps;
    };
    vector<seq_t> seq;
    for (auto seg:segs)
    {
        if (seg.a.x>seg.b.x+eps) swap(seg.a,seg.b);
        seq.push_back({seg.a.x,0,seg});
        seq.push_back({seg.b.x,1,seg});
    }
    sort(seq.begin(),seq.end(),seqcmp);
    point_t x_now;
    auto cmp=[&](const Segment &u, const Segment &v)
    {
        if (abs(u.a.x-u.b.x)<=eps || abs(v.a.x-v.b.x)<=eps) return u.a.y<v.a.y-eps;
        return ((x_now-u.a.x)*(u.b.y-u.a.y)+u.a.y*(u.b.x-u.a.x))*(v.b.x-v.a.x)<((x_now-v.a.x)*(v.b.y-v.a.y)+v.a.y*(v.b.x-v.a.x));
    };
    multiset<Segment,decltype(cmp)> s{cmp};
    for (const auto [x,o,seg]:seq)
    {
        x_now=x;
        const auto it=s.lower_bound(seg);
        if (o==0)
        {
            if (it!=s.end() && seg.is_inter(*it)) return true;
            if (it!=s.begin() && seg.is_inter(*prev(it))) return true;
            s.insert(seg);
        }
        else
        {
            if (next(it)!=s.end() && it!=s.begin() && (*prev(it)).is_inter(*next(it))) return true;
            s.erase(it);
        }
    }
}

```

```

    }
}
return false;
}

// 多边形面积并
// 轮廓积分, 复杂度  $O(n^2 \log n)$ ,  $n$  为边数
// ans[i] 表示被至少覆盖了  $i+1$  次的区域的面积
vector<long double> area_union(const vector<Polygon> &polys)
{
    const size_t siz=polys.size();
    vector<vector<pair<Point,Point>>> segs(siz);
    const auto check=[](const Point &u,const Segment &e){return !((u<e.a && u<e.b) || (u>e.a &&

auto cut_edge=[&](const Segment &e,const size_t i)
{
    const Line le{e.a,e.b-e.a};
    vector<pair<Point,int>> evt;
    evt.push_back({e.a,0}); evt.push_back({e.b,0});
    for (size_t j=0;j<polys.size();j++)
    {
        if (i==j) continue;
        const auto &pj=polys[j];
        for (size_t k=0;k<pj.p.size();k++)
        {
            const Segment s={pj.p[k],pj.p[pj.nxt(k)]};
            if (le.toleft(s.a)==0 && le.toleft(s.b)==0)
            {
                evt.push_back({s.a,0});
                evt.push_back({s.b,0});
            }
            else if (s.is_inter(le))
            {
                const Line ls{s.a,s.b-s.a};
                const Point u=le.inter(ls);
                if (le.toleft(s.a)<0 && le.toleft(s.b)>=0) evt.push_back({u,-1});
                else if (le.toleft(s.a)>=0 && le.toleft(s.b)<0) evt.push_back({u,1});
            }
        }
    }
    sort(evt.begin(),evt.end());
    if (e.a>e.b) reverse(evt.begin(),evt.end());
    int sum=0;

```



```

    for (size_t i=0;i<evt.size();i++)
    {
        sum+=evt[i].second;
        const Point u=evt[i].first,v=evt[i+1].first;
        if (!(u==v) && check(u,e) && check(v,e)) segs[sum].push_back({u,v});
        if (v==e.b) break;
    }
};

for (size_t i=0;i<polys.size();i++)
{
    const auto &pi=polys[i];
    for (size_t k=0;k<pi.p.size();k++)
    {
        const Segment ei={pi.p[k],pi.p[pi.nxt(k)]};
        cut_edge(ei,i);
    }
}
vector<long double> ans(siz);
for (size_t i=0;i<siz;i++)
{
    long double sum=0;
    sort(segs[i].begin(),segs[i].end());
    int cnt=0;
    for (size_t j=0;j<segs[i].size();j++)
    {
        if (j>0 && segs[i][j]==segs[i][j-1]) segs[i+(++cnt)].push_back(segs[i][j]);
        else cnt=0,sum+=segs[i][j].first^segs[i][j].second;
    }
    ans[i]=sum/2;
}
return ans;
}

```

// 圆面积并

// 轮廓积分，复杂度 $O(n^2 \log n)$

// ans[i] 表示被至少覆盖了 i+1 次的区域的面积

```

vector<long double> area_union(const vector<Circle> &circs)
{
    const size_t siz=circs.size();
    using arc_t=tuple<Point,long double,long double,long double>;
    vector<vector<arc_t>> arcs(siz);
    const auto eq=[](const arc_t &u,const arc_t &v)

```

```

{
    const auto [u1,u2,u3,u4]=u;
    const auto [v1,v2,v3,v4]=v;
    return u1==v1 && abs(u2-v2)<=eps && abs(u3-v3)<=eps && abs(u4-v4)<=eps;
};

auto cut_circ=[&](const Circle &ci,const size_t i)
{
    vector<pair<long double,int>> evt;
    evt.push_back({-PI,0}); evt.push_back({PI,0});
    int init=0;
    for (size_t j=0;j<circs.size();j++)
    {
        if (i==j) continue;
        const Circle &cj=circs[j];
        if (ci.r<cj.r-eps && ci.relation(cj)>=3) init++;
        const auto inters=ci.inter(cj);
        if (inters.size()==1) evt.push_back({atan2l((inters[0]-ci.c).y,(inters[0]-ci.c).x),0});
        if (inters.size()==2)
        {
            const Point dl=inters[0]-ci.c,dr=inters[1]-ci.c;
            long double argl=atan2l(dl.y,dl.x),argr=atan2l(dr.y,dr.x);
            if (abs(argl+PI)<=eps) argl=PI;
            if (abs(argr+PI)<=eps) argr=PI;
            if (argl>argr+eps)
            {
                evt.push_back({argl,1}); evt.push_back({PI,-1});
                evt.push_back({-PI,1}); evt.push_back({argr,-1});
            }
            else
            {
                evt.push_back({argl,1});
                evt.push_back({argr,-1});
            }
        }
    }
    sort(evt.begin(),evt.end());
    int sum=init;
    for (size_t i=0;i<evt.size();i++)
    {
        sum+=evt[i].second;
        if (abs(evt[i].first-evt[i+1].first)>eps) arcs[sum].push_back({ci.c,ci.r,evt[i].first,evt[i+1].first});
        if (abs(evt[i+1].first-PI)<=eps) break;
    }
}

```

```

    }
};

const auto oint=[](const arc_t &arc)
{
    const auto [cc,cr,l,r]=arc;
    if (abs(r-l-PI-PI)<=eps) return 2.01*PI*cr*cr;
    return cr*cr*(r-l)+cc.x*cr*(sin(r)-sin(l))-cc.y*cr*(cos(r)-cos(l));
};

for (size_t i=0;i<circs.size();i++)
{
    const auto &ci=circs[i];
    cut_circ(ci,i);
}
vector<long double> ans(siz);
for (size_t i=0;i<siz;i++)
{
    long double sum=0;
    sort(arcs[i].begin(),arcs[i].end());
    int cnt=0;
    for (size_t j=0;j<arcs[i].size();j++)
    {
        if (j>0 && eq(arcs[i][j],arcs[i][j-1])) arcs[i+(++cnt)].push_back(arcs[i][j]);
        else cnt=0,sum+=oint(arcs[i][j]);
    }
    ans[i]=sum/2;
}
return ans;
}

```